



LLM10:2026 Improper Output Handling

Model output is untrusted input

What changed in 2026

#5 → #10

Lower rank, wider scope:
now includes insecure AI-generated code

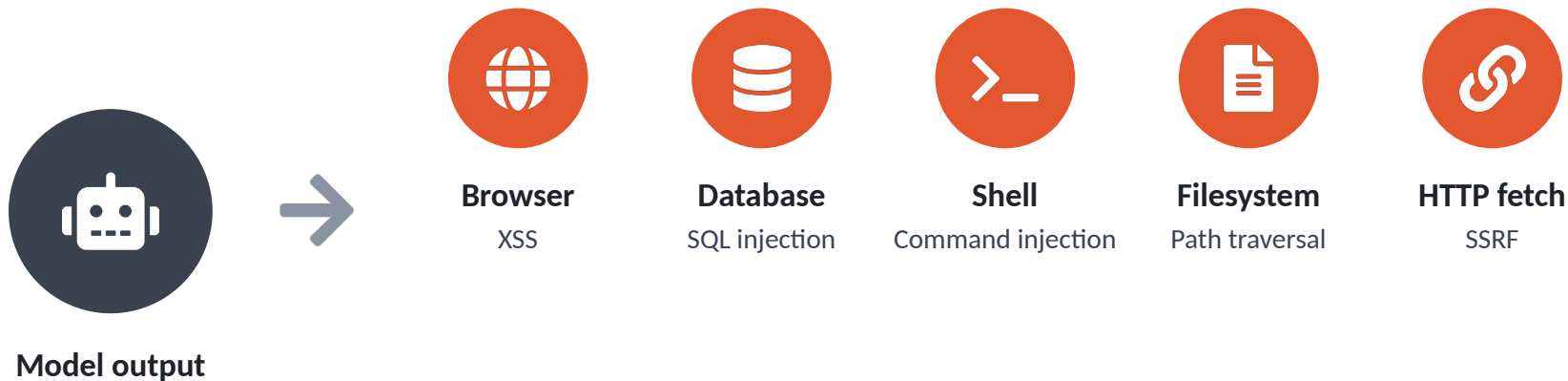
“The vulnerability is not the hallucination; it is what your application does with a plausible-looking string.”

OWASP / Giskard



Hiring signal DeepMind Agentic Red Team • OpenAI Security Engineer, Agent Security (runtime sandboxing, policy enforcement)

Every output lands in a sink



No new technique. Classic web bugs, with the model as the untrusted user. The category exists because developers skip escaping for model output.

Why an input classifier isn't enough

Text-to-SQL bot: “our injection classifier makes generated SQL safe”



No attack needed

The model can write harmful SQL by mistake



Classifiers get bypassed

“The Attacker Moves Second”:
most of 12 defenses >90%
bypass; all fell to human red-
teamers



Root flaw

The executor trusts model output

CVE-2026-22813: OpenCode

Coding agent • CVSS 9.4 • Jan 2026



Model output

Markdown → HTML,
unescaped, no CSP

XSS

Script runs in the localhost UI

Command API

Page can call the agent's API

Local RCE

Commands run on your
machine

Fix: escape for the context (HTML) • CSP • authenticate the local API

XSS: text shown as code

Model reply contains:

```
<script>steal()</script>
```



Escaped

Browser displays the characters. Harmless text.



Rendered raw

Browser runs steal() with the page's permissions.

Limit what bad SQL can do



Read-only role

Can't write or drop



Row-level security

Identity from the session, never from model args



Restricted views

Only the columns the bot needs



Timeout + row cap

Bounds runaway queries

Model chooses, code builds



Schema $\neq$ safe content

"filter": "1=1; DROP TABLE users" passes



Enums for structure

Bind parameters for values

Identifiers can't be bound

```
# table/col/op come from YOUR lookup tables
sql = f"SELECT * FROM {TABLES[t]} WHERE {COLS[c]} {OPS[op]} %s"
cur.execute(sql, (value,))
# value is data, never parsed as SQL
```

Tool arguments are a sink too



`read_file(path)`

Path traversal

`../../../../etc/passwd`

Resolve path, then check it's
inside docs



`fetch_url(url)`

SSRF

`169.254.169.254`

Resolve host, block internal IPs,
or allow-list domains



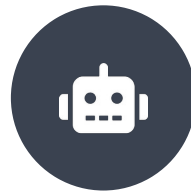
`run_cmd(cmd)`

Command injection

`x; rm -rf /`

No shell: argument list +
command allow-list

SSRF: your server fetches for them



Injected text

"fetch 169.254.169.254/..."

Your fetch_url tool

Runs inside your network

Metadata service

Returns cloud credentials

Model

Sends them back out

Resolve first: evil.com can point to an internal IP. Then block internal ranges, or allow-list domains.

Path checks miss tricks

```
realpath(join("/app/docs", p))
```

collapses .. and follows symlinks

...but it still misses:

- Encoding: %2e%2e%2f, double-encoding
- Unicode look-alikes, null bytes
- OS quirks: ..\, short names, case
- Hard links: a second name for the same file



TOCTOU

Time-of-check to time-of-use

1. Check: path is inside docs
2. Attacker swaps in a symlink
3. Open: reads /etc/passwd

The check was right; the file changed.

Make the bad target unreachable



Kernel enforces it

openat2 + RESOLVE_BENEATH: the OS refuses escapes while opening. No gap.



Sandbox the tool

Only /app/docs is mounted. /etc/passwd isn't the real one, or isn't there.



Sandbox generated code

AI-written code runs isolated, never in the app's process.

Contain what you can't fully validate.

Defense map by sink

Sink	Check (first layer)	Structural fix
Browser	Escape for context	CSP • auth on local APIs
SQL	Enums for identifiers	Bind params • read-only role • RLS
Shell	Command allow-list	Argument list, no shell • sandbox
Files	realpath + inside-check	RESOLVE_BENEATH • mount only docs
HTTP	Resolve + block internal IPs	Domain allow-list • egress filter

Interview takeaways

- Treat model output like user input: escape, bind, validate in the tool
- Classifiers detect; the executor must enforce
- Model chooses from options and supplies data; code builds the structure
- Checks miss tricks; containment doesn't